**Universidade do Minho**
Escola de Engenharia
Licenciatura em Engenharia Informática

# Unidade Curricular de Sistemas Operativos

Ano Letivo de 2024/2025
Grupo 64

**Carlos Silva**
a106792

**Francisco Soares**
a106882

**Nuno Soares**
a107366

SO

# Índice

**1.Introdução**

**1.1 Breve descrição do projeto**

**1.2 Objetivos principais**

**1.3 Tecnologias e mecanismos utilizados**

**2. Ficheiros do projeto**

**2.1 Ficheiros Principais**

**2.2 Scripts de Suporte**

**2.3 Outros Ficheiros**

**3. Arquitetura do programa**

**3.1 Estrutura de processos**

**3.2 Paralelização na Pesquisa**

**3.3 Armazenamento de dados**

**4. Análise de performance**

**5. Exemplo de comunicação cliente/server para facilitar a compreensão**

**6. Desafios e soluções**

**7.Conclusão**

# 1.Introdução

O nosso grupo desenvolveu um sistema cliente-servidor para indexação e pesquisa de documentos textuais, implementando um serviço robusto que permite aos utilizadores gerir e pesquisar eficientemente grandes volumes de documentos. Este projeto foi desenvolvido no âmbito da unidade curricular de Sistemas Operativos, com o objetivo de aplicar e consolidar conhecimentos sobre comunicação entre processos e gestão de recursos em ambiente Linux.

## 1.1.Breve descrição do projeto

A nossa solução divide-se num programa servidor – dserver – que gerencia a indexação, pesquisa e persistência dos metadados dos documentos, e num programa cliente – dclient – que fornece uma interface de linha de comandos para o utilizador enviar pedidos (indexar, consultar, remover ou pesquisar documentos). Os pedidos são enviados usando FIFOs nomeados e cada cliente cria um FIFO único para receber a resposta; assim, o servidor responde individualmente a cada cliente.

## 1.2 Objetivos Principais

Objetivos Principais
-Desenvolver um sistema de comunicação entre processos de forma eficiente, através da utilização de pipes nomeados.
-Garantir a persistência dos metadados em disco (no nosso caso no arquivo metadata.bin), permitindo a recuperação do estado mesmo após reiniciar o servidor.
-Implementar uma cache em memória baseada no algoritmo LRU para reduzir o custo de acesso aos dados frequentemente consultados.
-Implementar pesquisa de conteúdo de documentos, com suporte à paralelização quando indicado (opção -s com mais de um processo) **.**

## 1.3 Tecnologias e Mecanismos Utilizados

Na implementação do nosso projeto, recorremos exclusivamente a mecanismos do sistema operativo Linux, como nos era pedido no enunciado. As principais tecnologias e mecanismos utilizados incluem:
-Comunicação via FIFOs: O servidor cria um FIFO fixo para pedidos (FIFO_NAME) e cada cliente cria um FIFO exclusivo (baseado no seu PID) para receber a resposta.
-Gestão de Processos: Realizamos operações com fork() nos casos em que a pesquisa paralela é requerida (opção -s com nr_processes > 1). Para os comandos que alteram o estado global (ADD, DEL, SHUTDOWN, etc.), eliminamos o fork para serem processados diretamente no processo pai.
-Persistência e Acesso a Dados: Os metadados são gravados num ficheiro binário (metadata.bin) através de chamadas de sistema (open, read, write).
-Cache LRU: Implementamos uma cache em memória (modulada nos ficheiros cache.c e cache.h) utilizando uma lista duplamente ligada para manter a ordem de acesso e eliminar a entrada menos recentemente usada quando a capacidade é atingida.
-Pesquisa Paralela: A função de pesquisa divide o conjunto de documentos entre vários processos filhos (através da estrutura SearchTask), usando pipes anónimos e select() para agregar os resultados no processo pai.

# 2. Ficheiros do projeto

## 2.1 Ficheiros Principais

-dclient.c/.h Sistema cliente que processa comandos da linha de comandos e comunica com o servidor através de FIFOs.
-dserver.c/.h Servidor principal que gere indexação, pesquisa e mantém o estado global do sistema, processando pedidos dos clientes.
-handlers.c/.h Implementa handlers para cada comando (ADD, GET, DEL, COUNT, SEARCH), processando pedidos e gerando respostas.
-document.c/.h Gere metadados dos documentos, incluindo persistência em disco e operações básicas.
-cache.c/.h Implementa cache LRU para otimizar acesso a documentos frequentemente consultados.
-search.c/.h Módulo de pesquisa que implementa funcionalidades de busca sequencial e paralela em documentos.
-utils.c/.h Funções utilitárias para manipulação de ficheiros, strings e pesquisa de palavras em texto.
-common.h Define constantes, estruturas e tipos partilhados entre todos os módulos do programa.

## 2.2 Scripts de Suporte

-test_performance.sh Script para testes automatizados de performance, avaliando pesquisa paralela e impacto da cache.
-addGdatasetMetadata.sh Script para indexação em massa de documentos a partir de um catálogo TSV (Script fornecido pelos professores).

## 2.3 Outros Ficheiros

-Makefile Configura compilação do projeto, definindo dependências e regras de construção.
metadata.bin Ficheiro binário que mantém persistência dos metadados entre execuções do servidor (Se não existir é criado ao dar make e depois iniciar o servidor com os metadados)
-README.txt Documentação com instruções de configuração, compilação e execução do sistema.

# 3.Arquitetura do sistema

## 3.1 Estrutura de processos

O nosso sistema assenta numa arquitetura cliente-servidor, onde implementámos uma solução de comunicação entre processos que permite gerir eficientemente as operações de indexação e pesquisa de documentos. A estrutura fundamental divide-se em dois componentes principais:
-Servidor (dserver)
– Mantém em memória o estado global (array de documentos, contador de documentos, próxima chave, cache).
– Processa pedidos via FIFO; agora, os comandos que modificam o estado (ADD, DEL, SHUTDOWN, etc.) são tratados diretamente no processo pai, garantindo que as variáveis globais sejam atualizadas de forma consistente.
– Para a pesquisa de documentos (opção -s), se o número de processos solicitado for maior que 1, o servidor invoca a função handle_search_command que realiza forks internamente para dividir o trabalho entre os processos e agrega os resultados via pipes e select().

-Cliente (dclient)
– Lê os argumentos da linha de comandos, constrói o pedido (incluindo o FIFO de resposta exclusivo) e envia a mensagem estruturada ao FIFO de pedidos.
– Aguarda a resposta no seu próprio FIFO e imprime a informação no standard output do utilizador, depois libertando o canal de resposta (unlink)

## 3.2 Paralelização na Pesquisa

Desenvolvemos um mecanismo de pesquisa paralela que divide o trabalho entre N processos filhos. E é esta estrutura SearchTask que encapsula os parâmetros necessários:

```
typedef struct {
        int   start_idx;    // índice inicial no array
        int   end_idx;      // índice final no array
        char  keyword[MAX_BUF];  // palavra a pesquisar
        int  *results;      // buffer para resultados
        int   results_size;  // tamanho do buffer
        Document *docs;          // ponteiro para documentos
} SearchTask;
```

## 3.3 Armazenamento de dados

A estrutura do nosso arquivo de metadados inclui:

```
// Formato do metadata.bin
struct {
        int doc_count;              // número total de documentos
        int next_key;               // próxima chave disponível
        Document documents[doc_count];        // array de documentos
};
```

Cache LRU
Implementamos uma cache LRU utilizando uma lista duplamente ligada:

```
typedef struct CacheEntry {
        Document doc;           // metadados do documento
        struct CacheEntry *next;  // próxima entrada
        struct CacheEntry *prev;  // entrada anterior
} CacheEntry;
```

A nossa política de substituição que decidimos aplicar remove sempre a entrada menos recentemente utilizada quando a cache atinge sua capacidade máxima:

```
if (cache.size < cache.max_size) {
        new_entry = &cache.entries[cache.size++];
} else {
        new_entry = cache.tail;  // reutiliza entrada mais antiga
        cache.tail = cache.tail->prev;
}
```

## 4. Análise de performance

O nosso grupo desenvolveu um script de testes automatizado (test_performance.sh) que avalia o desempenho do sistema em dois aspectos principais:
Pesquisa Paralela: Testamos a palavra-chave "King" com diferentes números de processos (1, 2, 4, 8, 16, 64, 128, 256)
Cache: Avaliamos diferentes tamanhos de cache (10, 50, 100, 200, 1000, 2048)
Cada configuração foi testada com 5 iterações para obter médias representativas:
Análise dos Resultados
*Pesquisa Paralela*
*Processos,Tempo Médio*
*1,.0742933858*
*2,1.0459013014*
*4,.8477703778*
*8,.7822831884*
*16,.7757612256*
*64,.4838237772*
*128,.4860592344*
*256,.4156833646*

A pesquisa paralela mostrou-se eficiente, com tempos médios de execução decrescentes à medida que o número de processos aumentava. No entanto, notamos um aumento inesperado no tempo médio com 2 processos, que foi mais lento que a execução sequencial (1 processo).
Isto pode ser atribuído a vários fatores, incluindo o overhead de criação de processos e a comunicação entre eles. A pesquisa com 256 processos foi a mais rápida, com um tempo médio de 0.4156833646 segundos. Já a pesquisa com 1 processo (sequencial) teve um tempo médio de 0.0742933858 segundos, indicando que a paralelização se torna vantajosa apenas com um número suficientemente elevado de processos. Notamos ainda que a utilização de 4, 8 e 16 processos resultou em tempos médios semelhantes, demonstrando que a divisão de trabalho não trouxe ganhos significativos até ultrapassar os 64 processos.
Com 64 processos, o tempo médio registrado foi de 0.4838237772 segundos, representando uma melhora em relação à execução sequencial, e a partir de 128 processos o desempenho estabilizou em torno de 0.4860592344 segundos.

Fatores que influenciam estes resultados:
- Overhead de criação de processos
- Comunicação via pipes entre processos
- Fragmentação do trabalho em porções muito pequenas

*Impacto da Cache*
*CacheSize,Tempo Médio*
*10,.0687610534*
*50,.0688390300*
*100,.0677181638*
*200,.0669672112*
*1000,.0660386104 (melhor desempenho)*
*2048,.0660975066*

Ou seja:
-Com cache pequena (10 entradas): 0.0687610534s
-Com cache média (100 entradas): 0.0677181638s
-Com cache grande (2048 entradas): 0.0660975066s (melhor desempenho)

Esta melhoria significativa de desempenho deve-se a:
- Maior hit rate na cache
- Menos necessidade de recarregar documentos do disco
- Melhor aproveitamento da memória disponível

Embora não tenhamos observado grandes diferenças entre caches de 1000 e 2048 entradas, a cache de 1000 entradas teve o melhor desempenho com 0.0660386104 segundos. Isso sugere que um tamanho de cache maior não necessariamente resulta em melhor desempenho, possivelmente devido ao overhead de gestão da cache.

## 5. Exemplo de comunicação cliente/server para facilitar a compreensão

-O cliente cria e utiliza um FIFO fixo para enviar o pedido e gera um FIFO exclusivo (baseado no PID) para receber a resposta.
-O servidor fica em standby a aguardar pedidos no FIFO fixo, lê a mensagem, processa e responde através do FIFO exclusivo indicado na mensagem.
-Se o comando recebido for SHUTDOWN, o servidor envia a resposta de desligamento e encerra sua execução.

Agora, uma descrição cronológica de como a comunicação acontece:
1. Servidor em standby
-O servidor é iniciado (executa o programa dserver).
-Cria o FIFO fixo (com o nome definido em FIFO_NAME, por exemplo, "/tmp/doc_service") e entra em um loop de espera, abrindo-o em modo leitura para aguardar mensagens dos clientes.

2. Cliente prepara e envia o Pedido
O usuário executa o cliente com os parâmetros que quiser (como -a, -c, etc.).
O cliente abre o FIFO fixo ("/tmp/doc_service") em modo escrita para enviar seu pedido.
Antes de enviar, o cliente cria um FIFO exclusivo para resposta, usando um nome que incorpore seu PID (por exemplo, "/tmp/doc_service_resp_12345").
O cliente constrói uma estrutura de mensagem (ClientMessage) contendo:
- O seu PID;
- O nome do FIFO exclusivo para resposta;
- A string de comando (por exemplo, "ADD title|authors|year|path").
Essa estrutura é escrita no FIFO fixo e logo de seguida o cliente fecha o descritor de escrita.

3. Servidor lê e processa o pedido
O servidor, que está constantemente a ler o FIFO fixo, deteta o novo pedido.
De seguida, lê a estrutura ClientMessage, identifica o comando (através de strtok e comparações) e invoca o handler apropriado (como handle_add para ADD, handle_get para GET, etc.).
No processamento, se o comando for SHUTDOWN, o servidor limpa os recursos e encerra; caso contrário, ele compõe a resposta (por exemplo, "Document 1 indexed" ou metadados do documento) em um buffer.

4. Servidor envia a resposta
Após compor a resposta, o servidor abre o FIFO exclusivo informado pelo cliente (por exemplo, "/tmp/doc_service_resp_12345") em modo escrita.
Escreve a resposta no FIFO exclusivo e fecha esse canal.

5. Cliente recebe e processa a resposta
Enquanto isso, após enviar o pedido, o cliente abre seu FIFO exclusivo em modo leitura.
Ele lê a resposta enviada pelo servidor.
A leitura continua até que a função read retorne 0 (EOF), indicando que não há mais dados para ler.
Após concluir a leitura, o cliente fecha o FIFO exclusivo e executa o unlink para removê-lo.

## 6. Desafios e soluções

Um dos problemas mais críticos que identificámos foi relacionado com a consistência do estado global do sistema. Inicialmente, a nossa implementação utilizava fork() para cada pedido recebido, o que resultava em problemas graves de consistência. As atualizações de variáveis críticas como next_key e doc_count ocorriam em processos filhos, não se refletindo no processo pai, levando a inconsistências no estado global do servidor.

Para resolver este problema, implementamos uma solução que limita o uso de fork() exclusivamente para operações de pesquisa paralela. Todos os outros comandos que modificam o estado global (como ADD, DEL, SHUTDOWN) passaram a ser processados diretamente no processo pai, garantindo assim a consistência dos dados.

Também tivemos problemas em pesquisa paralela, identificamos situações de encravamento quando o número de processos solicitados era superior ao número de documentos disponíveis. Este problema manifestava-se quando tínhamos pipes que não eram fechados corretamente, e que depois causavam bloqueios indefinidos. Também haviam processos sem trabalho que mantinham recursos alocados desnecessariamente, e às vezes o processo pai ficava à espera de resultados que nunca chegariam.

Para solucionar estas questões, implementámos algumas melhorias, desenvolvemos um sistema de gestão de pipes com uma função cleanup_pipes que garantisse o fecho adequado de todos os descritores, melhoramos a gestão de processos com contagem precisa de processos ativos e libertação adequada de recursos, e por fim utilizamos a distribuição de trabalho com verificação prévia da necessidade real de criar novos processos e distribuição equilibrada de documentos entre os processos disponíveis.

## 7. Conclusão

Para concluir, o nosso trabalho cumpre corretamente os cumpre os requisitos do enunciado, os programas dclient e dserver são escritos em C, comunicam entre si utilizando pipes nomeados , mantêm o estado global (com persistência em disco e cache em memória) e suportam operações de indexação, consulta, remoção e pesquisa – inclusive de forma paralela, quando solicitado, embora que existam certos procedimentos, nomeadamente ao nível do paralelismo, que ainda causam certa instabilidade ao correr no nosso programa.

Por fim, as alterações na modulação (processamento de comandos modificadores no processo pai) conseguiram garantir a consistência dos dados e o comportamento esperado ao adicionar e remover documentos, bem como ao dar shutdown ao server via comando -f.